

Technical Assessment — AI Sales & Customer Service Agent

Overview

Build an **AI Sales & Customer Service Agent** for a business of your choice.

You can choose any business domain, for example:

- E-commerce / online store
- Restaurant
- Pharmacy
- Travel agency
- Real estate
- Electronics store
- Car dealership
- Any other business you think is suitable

The goal is to build a working AI agent that can **answer customer questions, retrieve information using RAG, and perform business actions through tools/functions**.

1. AI Agent

The system must have an AI agent capable of handling:

Customer Service

The agent should be able to:

- Understand customer questions.
- Answer questions about the business.
- Retrieve relevant information from the business knowledge base.
- Handle basic customer-service conversations.
- Maintain enough conversation context to provide coherent responses.

Sales

The agent should be able to:

- Recommend relevant products/services.
- Answer questions about products/services.
- Provide prices and other available information.
- Help the customer make a decision.

- Perform at least one real business action.
-

2. RAG — Required

The agent **must use RAG**.

The knowledge base should contain information relevant to the business you selected.

For example, an e-commerce business could have:

- Products
- Prices
- Descriptions
- Categories
- FAQs
- Policies
- Delivery information

The agent should retrieve relevant information from this knowledge base when answering customers.

RAG Management

The system must provide a way to:

- Add knowledge/data.
- Update existing data.
- Delete data.
- Use the updated data in the agent's retrieval process.

The exact implementation of the vector database / retrieval system is up to you.

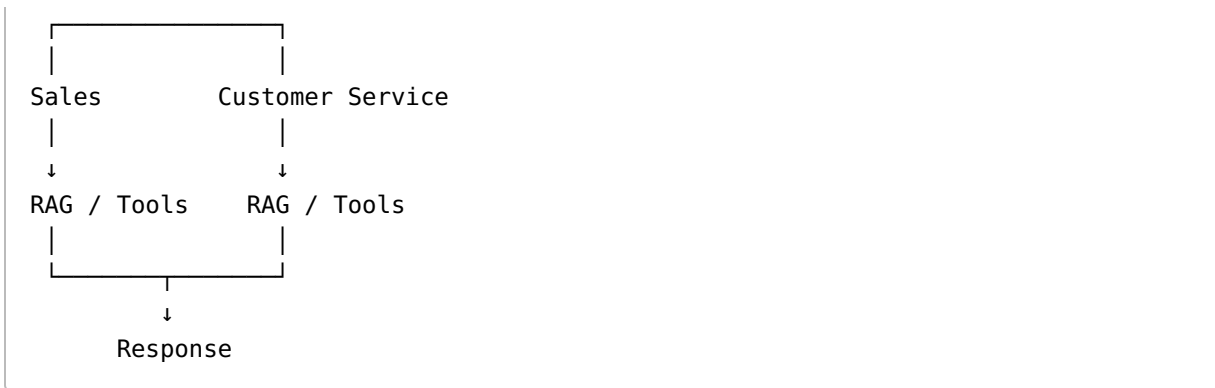
3. LangGraph — Required

The agent **must be implemented using LangGraph**.

The graph should have a meaningful workflow rather than simply calling an LLM once.

For example, your graph could include:

```
User Message
  ↓
Intent / Request Understanding
  ↓
```



You are free to design the architecture differently if you have a better approach.

4. Business Action / Function Calling — Required

The agent must be able to **perform an action using a tool/function**.

The action should make sense for the business you selected.

Examples:

E-commerce

- Create an order
- Add product to cart
- Check product availability

Restaurant

- Create a reservation
- Create an order
- Check menu availability

Travel Agency

- Create a booking request
- Search available trips
- Create a customer inquiry

Real Estate

- Create a property viewing request
- Create a customer lead

Car Dealership

- Create a test-drive request
- Create a sales lead

The action should actually interact with your backend/database rather than being a fake response from the LLM.

5. Flask Dashboard — Required

Build an administration dashboard using:

Flask + Flask Templates

The dashboard should allow an administrator to see and manage the system.

At minimum, the dashboard should include:

Business Data

Depending on your selected business, display relevant data such as:

- Products
- Services
- Orders
- Customers
- Leads
- Reservations

RAG Data Management

The administrator should be able to:

- View RAG data.
- Add new data.
- Edit existing data.
- Delete data.

The purpose is to demonstrate that the knowledge base can be managed without changing the source code.

6. Database

You must use a database with an **ORM**.

You are free to choose the database and ORM.

For example:

- PostgreSQL + SQLAlchemy
- SQLite + SQLAlchemy
- Another reasonable relational database + ORM

The database should contain the required business entities and agent-related data.

7. Required Technology Stack

Backend

- **Python**
- **Flask**
- **ORM**
- Database of your choice

AI

- **LangGraph — Required**
- **RAG — Required**
- LLM of your choice
- Embedding model of your choice
- Vector database / retrieval system of your choice

Frontend

- Flask Templates
- HTML
- CSS
- JavaScript where needed

You are free to use additional libraries when they are useful.

8. Bonus — Meta Integration

Bonus: Facebook Messenger

As an additional challenge, integrate the agent with a **Facebook Page / Meta Messenger**.

The flow should look approximately like:



This is **not required**, but it will be considered a strong bonus.

9. Deliverables

You must submit:

1. GitHub Repository

Submit a GitHub repository containing the complete project.

The repository should include:

- Source code
- Database models
- Agent / LangGraph implementation
- RAG implementation
- Flask dashboard
- Tool/function implementations
- Requirements/dependencies
- README

2. README

The README should explain:

- Business/domain you selected.
- Project architecture.
- How the LangGraph agent works.

- How RAG works.
- Database structure.
- Available tools/functions.
- How to run the project locally.
- Environment variables required.
- Example conversations.
- Any limitations or assumptions.

3. Demo

The project should be runnable and demonstrate:

1. Customer starts a conversation.
2. Agent understands the request.
3. Agent retrieves information using RAG.
4. Agent responds appropriately.
5. Agent performs a business action using a tool/function.
6. Administrator can view the result from the dashboard.
7. Administrator can update RAG/business data from the dashboard.

10. AI Usage Policy

You **may use AI tools** during the assessment.

However, **do not generate the entire project using AI and submit it without understanding it.**

We are evaluating your:

- Problem-solving ability
- Software engineering skills
- Understanding of AI agents
- Understanding of RAG
- Understanding of LangGraph
- Backend architecture
- Database design
- Ability to explain your own implementation

During the next interview, you may be asked to explain parts of your code and make changes to the system.

If you use AI-generated code, you are expected to understand, test, and be able to explain it.

11. Deadline

Submission Deadline: 20 September 2026

Submissions after the deadline may not be considered.

12. Evaluation Criteria

We will evaluate the project based on:

Area	Evaluation
Agent Architecture	Quality and structure of the agent
LangGraph	Proper and meaningful use of LangGraph
RAG	Retrieval quality and implementation
Function Calling	Correct implementation of business actions
Backend	Flask architecture and code quality
Database	ORM and database design
Dashboard	Functionality and usability
Code Quality	Clean, maintainable and organized code
Error Handling	Handling failures and edge cases
Documentation	README and setup instructions
Understanding	Ability to explain and modify the implementation
Bonus	Meta Messenger integration

Final Note

There is **no single correct solution**.

Choose a business that interests you and design the system around it.

We are more interested in **how you think, design, build, and explain the system** than in the specific business you choose.

If you pass this technical assessment, you will move to the **next stage: the final interview**.